GRADLE APP

Step1:Create new directory inside “devops” folder with name “MyGradleApp1”.

Initialize Gradle using the command **“gradle init --type java-application”**

```
vboxuser@prathwish03:~/devops/MyMavenToGradle$ cd
vboxuser@prathwish03:~$ cd devops
vboxuser@prathwish03:~/devops$ mkdir MyGradleApp1
vboxuser@prathwish03:~/devops$ cd MyGradleApp1
vboxuser@prathwish03:~/devops/MyGradleApp1$ ls
vboxuser@prathwish03:~/devops/MyGradleApp1$ gradle init --type java-application

BUILD SUCCESSFUL in 2s
2 actionable tasks: 2 executed
vboxuser@prathwish03:~/devops/MyGradleApp1$ ls
build.gradle  gradle  gradlew  gradlew.bat  settings.gradle  src
vboxuser@prathwish03:~/devops/MyGradleApp1$ tree
.
├── build.gradle
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── settings.gradle
└── src
    ├── main
    │   └── java
    │       └── App.java
    └── test
        └── java
            └── AppTest.java

7 directories, 8 files
```

Step2:Edit build.gradle using “gedit”

```
vboxuser@prathwish03:~/devops/MyGradleApp1$ gedit build.gradle

[1]+  Stopped                 gedit build.gradle
vboxuser@prathwish03:~/devops/MyGradleApp1$ gradle build

BUILD SUCCESSFUL in 7s
7 actionable tasks: 7 executed
```

/*

* This build file was generated by the Gradle 'init' task.

*

* This generated file contains a sample Java project to get you started.

* For more details take a look at the Java Quickstart chapter in the Gradle

* user guide available at https://docs.gradle.org/4.4.1/userguide/tutorial_java_projects.html

```
 */

// Apply the java plugin to add support for Java
apply plugin: 'java'

// Apply the application plugin to add support for building an application
apply plugin: 'application'

// In this section you declare where to find the dependencies of your project
repositories {
   // Use jcenter for resolving your dependencies.
   // You can declare any Maven/Ivy/file repository here.
   jcenter()
}

dependencies {
   // This dependency is found on compile classpath of this component and
consumers.
   compile 'com.google.guava:guava:23.0'

   // Use JUnit test framework
   testCompile 'junit:junit:4.12'
}

// Define the main class for the application
mainClassName = 'App'

task Hello{
doLast{
```

```
        println "This is DevOps Lab"
}
}
```

Step3:Folder structure can be seen using “tree” command

```
vboxuser@prathwish03:~/devops/MyGradleApp1$ tree
.
├── build
│   ├── classes
│   │   └── java
│   │       ├── main
│   │       │   └── App.class
│   │       └── test
│   │           └── AppTest.class
│   ├── distributions
│   │   ├── MyGradleApp1.tar
│   │   └── MyGradleApp1.zip
│   ├── libs
│   │   └── MyGradleApp1.jar
│   ├── reports
│   │   └── tests
│   │       └── test
│   │           ├── classes
│   │           │   └── AppTest.html
│   │           ├── css
│   │           │   ├── base-style.css
│   │           │   └── style.css
│   │           ├── index.html
│   │           ├── js
│   │           │   └── report.js
│   │           └── packages
│   │               └── default-package.html
│   ├── scripts
│   │   ├── MyGradleApp1
│   │   └── MyGradleApp1.bat
│   ├── test-results
│   │   └── test
│   │       ├── binary
│   │       │   ├── output.bin
│   │       │   ├── output.bin.idx
│   │       │   └── results.bin
│   │       └── TEST-AppTest.xml
│   └── tmp
│       ├── compileJava
│       ├── compileTestJava
│       └── jar
│           └── MANIFEST.MF
├── build.gradle
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── settings.gradle
└── src
    ├── main
    │   └── java
    │       └── App.java
    └── test
        └── java
            └── AppTest.java

29 directories, 26 files
```

Step4: Run the gradle using “gradle run” command.

```
vboxuser@prathwish03:~/devops/MyGradleApp1$ gradle run

> Task :run
Hello world.


BUILD SUCCESSFUL in 2s
2 actionable tasks: 1 executed, 1 up-to-date
vboxuser@prathwish03:~/devops/MyGradleApp1$ gedit build.gradle
[1]   Killed                  gedit build.gradle

[2]+  Stopped                 gedit build.gradle
vboxuser@prathwish03:~/devops/MyGradleApp1$ gradle Hello

> Task :Hello
This is DevOps Lab


BUILD SUCCESSFUL in 2s
1 actionable task: 1 executed
vboxuser@prathwish03:~/devops/MyGradleApp1$ gradle run

> Task :run
Hello world.


BUILD SUCCESSFUL in 2s
2 actionable tasks: 1 executed, 1 up-to-date
```

Step5: Can create ne task and run it using name of the task

```
vboxuser@prathwish03:~/devops/MyGradleApp1$ gradle Hello
Starting a Gradle Daemon, 1 incompatible and 1 stopped Daemons could not be reused, use --status for details

> Task :Hello
This is DevOps Lab


BUILD SUCCESSFUL in 29s
1 actionable task: 1 executed
vboxuser@prathwish03:~/devops/MyGradleApp1$
```

Gradle pipeline

Step 1: Create new file named "**Jenkinsfile**" inside MyGradleApp folder and push into git

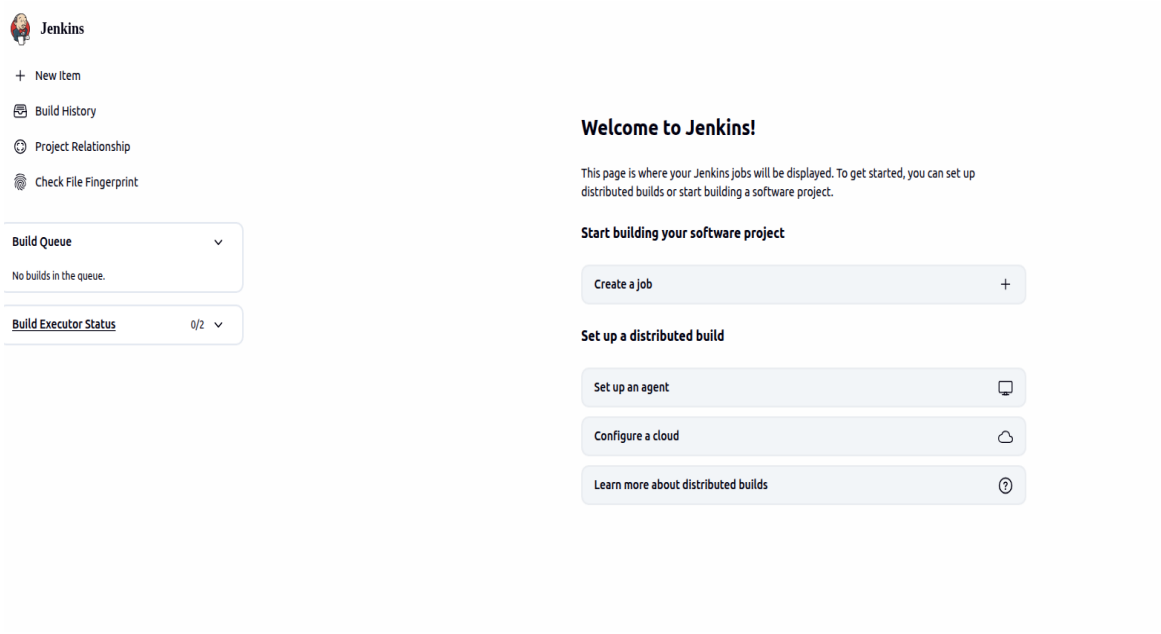
Jenkinsfile:

```
pipeline {
   agent any  // Use any available agent

   tools {
      gradle 'Gradle'  // Ensure this matches the name configured in Jenkins
      jdk 'JDK'
   }
   stages {
      stage('Checkout') {
         steps {
            git branch: 'master', url:
'https://github.com/PrathwishShetty03/MyGradleApp.git'
         }
      }

      stage('Build') {
         steps {
            sh 'gradle build'  // Run Gradle build
         }
      }

      stage('Test') {
         steps {
            sh 'gradle test'  // Run unit tests
         }
      }

      stage('Run Application') {
         steps {
            // Start the JAR application
```

```
            sh 'gradle run'
        }
    }
  }

  post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
  }
}
```

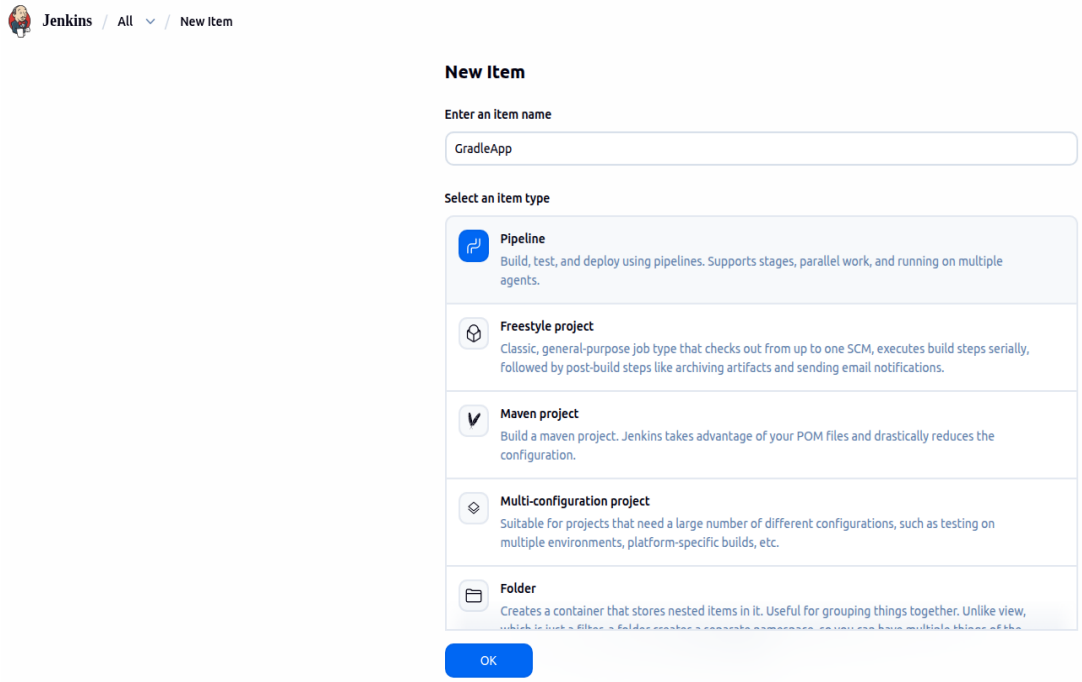
Step 2:Open Jenkins Dashboard

- Open Jenkins in browser
- Click on **New Item**



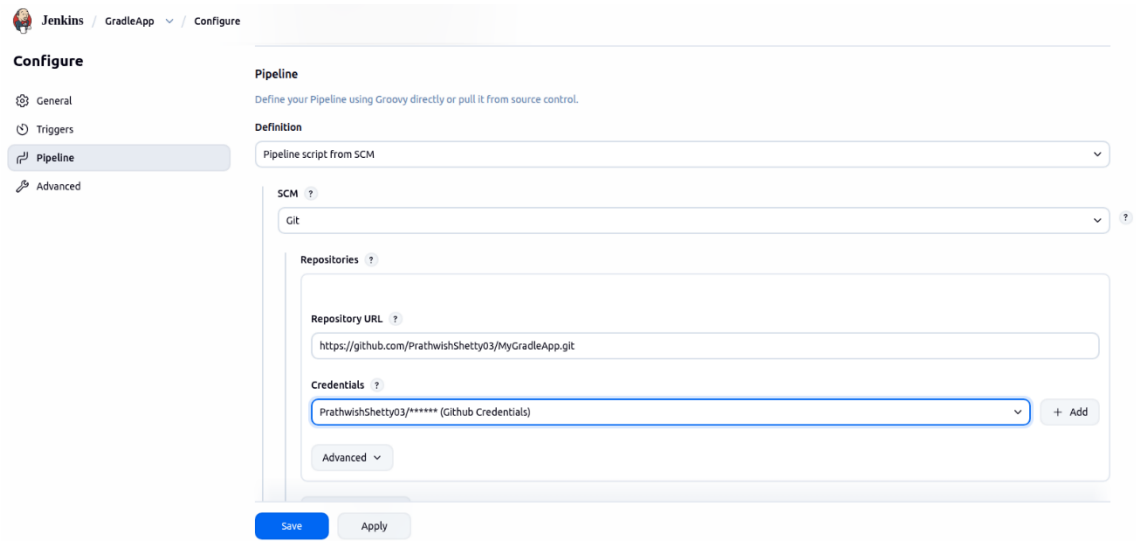
Step 3: Create Pipeline Job

- Enter name: **GradleApp**
- Select **Pipeline**
- Click **OK**
-



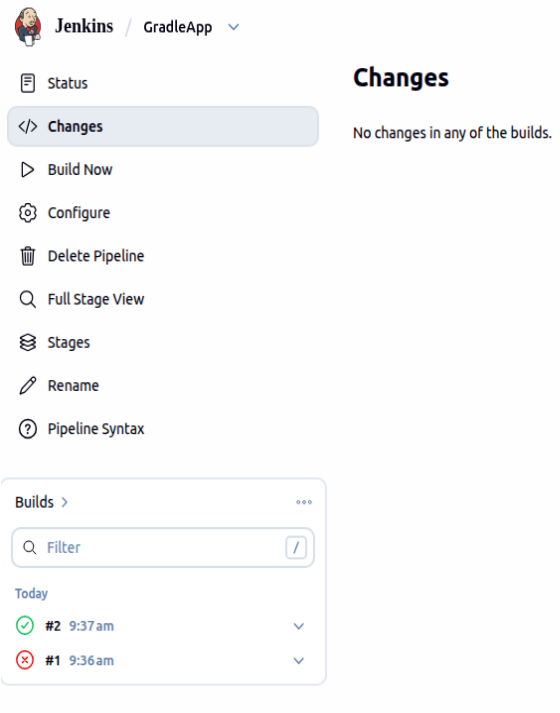
Step 4: Configure Pipeline with GitHub

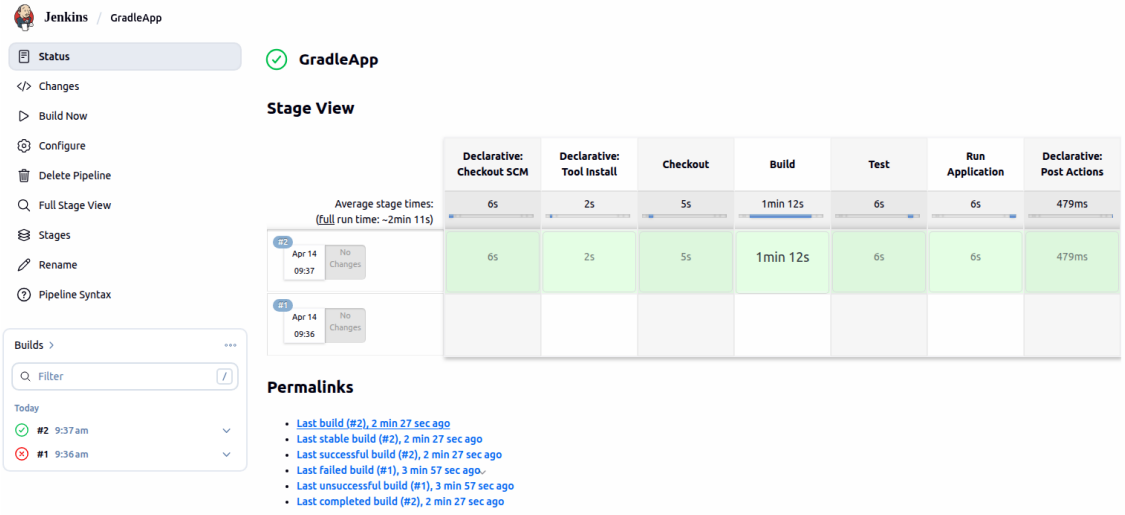
- Select **Pipeline script from SCM**
- SCM → Git
- Enter repository URL
- Add credentials
- Click **Save**



Step 5: Run the Pipeline

- Click **Build Now**



Step 6: Look for the status of project

- Check **Stage View**
- View **Build History**